Grado en Ingeniería Informática

Fundamentos de Programación II 25/26

Hito 2

Proyecto práctico: Hito 2

Resumen

El objetivo de esta práctica es afianzar los conocimientos adquiridos sobre creación y manejo de clases y objetos, aplicando un flujo completo de análisis, diseño e implementación orientado a objetos. El trabajo se plantea por parejas y se centra en la construcción del núcleo de un videojuego RPG (Role-Playing Game).

1. Objetivos

- Aplicar los fundamentos de programación orientada a objetos en un caso práctico realista.
- Diseñar e implementar un sistema modular con clases, relaciones y responsabilidades bien definidas.
- Practicar el uso de arrays de tamaño fijo y lógica de control de flujo en métodos.
- Consolidar el proceso de trabajo por fases: análisis, diseño e implementación.

2. Normas de trabajo

El ejercicio se realizará por parejas y se entregará en la tarea habilitada en Campus Virtual. Se deberán seguir estas normas:

- **Entrega**
  - Todos los archivos deberán comprimirse en un único archivo ZIP.
  - El nombre del ZIP debe seguir el formato `LX_XX.zip` según el grupo de Campus Virtual.
  - El ZIP deberá incluir:
    - Imagen del diagrama de clases y relaciones UML (`.png` o `.jpg`).
    - Únicamente la carpeta `src` con los archivos `.java`.
- **Restricciones de implementación**
  - No utilizar múltiples `return` en una misma función.
  - No utilizar `continue`.
  - No utilizar `break`, salvo en sentencias `switch`.
  - No utilizar `ArrayList`.
- Queda totalmente **prohibido el uso de Inteligencia Artificial** para realizar estas prácticas. Cualquier indicio de su utilización será penalizado.

3. Fases del desarrollo orientado a objetos

Se pide seguir las fases del desarrollo de software orientado a objetos:

- **Análisis:** En esta fase se identifican los requisitos del sistema. Intentamos encontrar los objetos y relaciones adecuados para el problema propuesto. Investigamos el problema centrándonos en los conceptos del dominio del problema, sin preocuparnos por la implementación. Es importante entender el problema y definir claramente los requisitos antes de pasar a la siguiente fase.
- **Diseño:** En esta fase se define la arquitectura del sistema. Partiendo de los objetos y relaciones identificados en el análisis, se diseña la estructura del sistema. Se definen las clases, sus atributos y métodos (aún sin pensar en la implementación, solo en aquella funcionalidad que cada clase debe tener), así como las relaciones entre ellas (herencia, composición, etc.). Es importante diseñar un sistema modular y flexible que permita cambios futuros sin afectar a toda la aplicación. En esta fase es fundamental el uso de diagramas UML (especialmente el diagrama de clases).
- **Implementación:** Únicamente cuando tenemos claro el diseño, pasamos a la implementación. En esta fase se escribe el código fuente de la aplicación siguiendo el diseño establecido. Es importante seguir buenas prácticas de programación y mantener el código limpio y organizado.

4. Desarrollo de la práctica

En este hito se amplía el núcleo del videojuego para poder trabajar con varios personajes. Los personajes se cargarán desde un fichero de entrada (`personajes.txt`) y el menú permitirá seleccionar con cuál de ellos queremos jugar. En esta versión, todos los personajes tendrán un inventario de tamaño fijo (5 posiciones), por lo que, al igual que en el Hito 1, deberéis tener cuidado con los huecos libres y con qué ocurre cuando el inventario está lleno.

Además, se introduce una tienda donde el personaje podrá comprar objetos. En esta fase no es necesario implementar la venta de objetos del personaje a la tienda. La tienda tendrá su propio inventario con la misma lógica que el personaje y con una capacidad máxima de 20 objetos: cuando un personaje compra un objeto, ese objeto pasa a su inventario y desaparece de la tienda. El catálogo inicial de la tienda se cargará desde un fichero de entrada (`tienda.txt`).

También se amplía el sistema de equipamiento. Además del arma, el personaje podrá equipar armaduras de tres tipos: cabeza, cuerpo y pies (por ejemplo, casco, coraza y botas; se deberá utilizar un `enum`). Cada pieza de equipo (arma o armadura) podrá aportar un bonus de ataque y/o de defensa. Para modelar esta relación se deberá utilizar una interfaz que represente el contrato del equipamiento, de modo que el personaje pueda tratar de forma uniforme el arma y las armaduras mediante polimorfismo.

En concreto, el personaje deberá almacenar su equipo en un único array de tamaño fijo de 4 posiciones, cuyo tipo sea el de la interfaz. Así, la interfaz deberá definir por contrato que todo objeto equipable proporcione funciones que devuelvan el bonus de ataque y de defensa que

aporta.

El menú se organizará por niveles: un menú principal, un menú del personaje y un menú de la tienda. Desde el menú principal se podrá listar a los personajes y entrar en el menú de uno concreto. En el menú del personaje se podrá ver su información, explorar el inventario, equipar un arma o una armadura, desequipar una posición e ir a la tienda. En el menú de la tienda se podrá ver el catálogo y realizar compras.

4.1. Ficheros de entrada

`personajes.txt`. El archivo de personajes se organiza en bloques:

- Inicio de bloque: `PERSONAJE;nombre;dinero`
- Líneas de equipo:
  `EQUIPO;ARMA;nombre;precio;peso;ataque;longitud;antiguedad;integridad`
  `EQUIPO;ARMADURA;nombre;precio;peso;tipo;defensa`
  `EQUIPO;VACIO;slot`
- Líneas de inventario:
  `INV;ARMA;nombre;precio;peso;ataque;longitud;antiguedad;integridad`
  `INV;ARMADURA;nombre;precio;peso;tipo;defensa`
  `INV;METAL;nombre;precio;peso;dureza;pureza;composicion`
- Fin de bloque: `FIN`

Ejemplo (`personajes.txt`):

```
PERSONAJE;Alba;150
EQUIPO;ARMA;EspadaCorta;50;3;12;95.5;3;88
EQUIPO;ARMADURA;CascoHierro;40;2;CABEZA;6
EQUIPO;ARMADURA;CorazaCuero;60;5;CUERPO;9
EQUIPO;VACIO;PIES
INV;ARMA;HachaGrande;80;6;18;120.0;12;74
INV;METAL;LingoteHierro;25;4;7;75;Fe
INV;METAL;LingoteCobre;20;3;4;80;Cu
FIN

PERSONAJE;Bran;95
EQUIPO;VACIO;ARMA
EQUIPO;ARMADURA;CascoCuero;18;1;CABEZA;2
EQUIPO;VACIO;CUERPO
EQUIPO;ARMADURA;BotasCuero;15;1;PIES;1
INV;METAL;LingoteBronce;30;3;5;68;Br
FIN
```

`tienda.txt`. Cada línea del fichero representa un objeto del catálogo:

- `ARMA;nombre;precio;peso;ataque;longitud;antiguedad;integridad`
- `ARMADURA;nombre;precio;peso;tipo;defensa`
- `METAL;nombre;precio;peso;dureza;pureza;composicion`

Ejemplo (`tienda.txt`):

```
ARMA;MandobleViejo;90;7;20;140.0;18;81
ARMA;LanzaCorta;55;4;11;180.0;6;90
ARMADURA;CascoAcero;70;3;CABEZA;8
ARMADURA;PetoHierro;110;8;CUERPO;14
ARMADURA;GrebasCuero;40;2;PIES;4
METAL;LingotePlata;60;2;5;92;Ag
METAL;LingoteTitanio;140;3;10;74;Ti
```